Step 2 — The RAG Layer and the Skill Extractor — Solution

> Back to the step · Hint 1 · Hint 2

Story: this builds the resume RAG store (chunk, embed, retrieve — Doc08's pattern) and the job-description skill extractor with structured output — the two real data sources the matcher from Step 1 actually needs. **If not:** without semantic search, a resume line like "built APIs in Python" would never match a plain "Python" requirement, and the whole point of using embeddings here would be lost.

The resume store

```
# src/rag/resume_store.py
# Chunk the resume, embed it, and store it in Chroma so a skill can be searched
# for by meaning. Uses Doc08's splitter and loaders stack.
from langchain_openai import OpenAIEmbeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter

import chromadb

COLLECTION_NAME = "resume"
EMBEDDINGS = OpenAIEmbeddings(model="text-embedding-3-small")


def build_resume_store(resume_text):
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=300, chunk_overlap=40,
    )
    chunks = splitter.split_text(resume_text)

    client = chromadb.Client()
    try:
        client.delete_collection(COLLECTION_NAME)
    except Exception:
        pass
    collection = client.create_collection(COLLECTION_NAME)

    ids = []
    for index in range(len(chunks)):
```

```
        ids.append(f"chunk-{index}")
    collection.add(
        ids=ids, documents=chunks,
        embeddings=EMBEDDINGS.embed_documents(chunks),
    )
    return collection
```

The retriever

```
# src/rag/retriever.py
# Return the distance of the resume chunk closest to a skill - lower means the
# resume covers it. A big number when the resume is empty.
from src.rag.resume_store import EMBEDDINGS

NO_MATCH_DISTANCE = 2.0


def best_distance(collection, skill):
    if collection.count() == 0:
        return NO_MATCH_DISTANCE
    results = collection.query(
        query_embeddings=[EMBEDDINGS.embed_query(skill)], n_results=1,
    )
    distances = results["distances"][0]
    if not distances:
        return NO_MATCH_DISTANCE
    return distances[0]
```

The job-description skill extractor

```
# src/extractor.py
# Pull the required skills out of a job description as a typed list.
from langchain_core.prompts import ChatPromptTemplate

from src.schemas import RequiredSkills

SKILLS_PROMPT = (
    "From the job description below, list the required skills and "
    "technologies as short names (e.g. Python, Docker, Leadership). Return "
    "an empty list if none are stated.\n\nJob description:\n{job}"
)
```

```
def extract_required_skills(model, job_text):
    prompt = ChatPromptTemplate.from_template(SKILLS_PROMPT)
    chain = prompt | model.with_structured_output(RequiredSkills)
    return chain.invoke({"job": job_text})
```

Why this is the "by meaning" part

- **The resume is embedded, so wording stops mattering.** A "Python" query lands close to a chunk that says "built REST APIs in Python" even though the words differ — that's the semantic match a keyword filter can't make.
- **`best_distance` returns one number per skill.** That's exactly what Step 1's `match_skills` needs — it doesn't care how the number was produced, only whether it's under the cutoff. The RAG layer and the decision engine stay cleanly separate.
- **The JD skills are typed.** `with_structured_output(RequiredSkills)` gives you a real list to loop over, not text to parse.
- **Tuning the cutoff** trades precision for recall: a tighter cutoff demands a closer match (fewer false matches, more false gaps); a looser one is the reverse. `MATCH_CUTOFF` makes it a setting, not a magic number buried in code.

The live run — real resume, real JD, real distances — is Step 3.